

Text Sentiment Classification Based on Convolutional Neural Networks

Changjin Li
231880264

Abstract

Text sentiment classification is an important task in natural language processing (NLP). Traditional feature-based methods usually suffer from sparse representations and limited semantic understanding. In this work, we study CNN-based sentiment classification by exploring the impact of different word representations and network architectures. The ablation study demonstrates that CNN effectively extracts local features, while combining RNN and CNN can further improve performance by capturing both local and global information.

1. Introduction

Text classification is a fundamental task in natural language processing (NLP), with applications on sentiment analysis. Traditional methods often rely on hand-crafted features such as n-grams, bag-of-words, or TF-IDF representations, which are sparse, high-dimensional, and fail to capture semantic relationships between words. However, convolutional neural networks (CNNs) have been shown to be effective for sentence classification tasks. In this work, we propose a simple yet effective hybrid embedding approach for CNN-based text classification.

2. Methods

Overall Architecture. The overall architecture of the proposed model is illustrated in Figure 1. Given an input sentence, it is first tokenized and mapped to a sequence of word indices. Each sentence is padded or truncated to a fixed length $L = 50$. An embedding layer transforms each word index into a dense vector of dimension $d = 300$. The embedding matrix is initialized with pre-trained word vectors for words or randomly. The embedded sentence is represented as a matrix of shape (L, d) , which is then fed into a convolutional neural network (CNN) module applying multiple convolution kernels of different window sizes. Each convolution operation is followed by a ReLU activation and a global max-pooling layer. The features extracted by all kernels are concatenated and passed through a dropout layer (dropout rate $p = 0.5$) to reduce overfitting. Finally, a fully

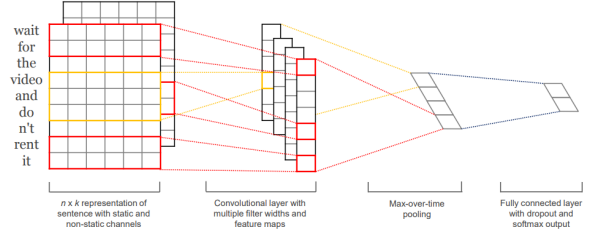


Figure 1. Model architecture with CNN layer. [3]

connected (FC) layer with softmax produces a probability distribution over the $C = 5$ classes. The overall architecture is trained end-to-end using the cross-entropy loss.

CNN. The CNN module consists of three parallel convolutional layers, each with $k \in [3, 4, 5]$ filter windows (kernel sizes). For each kernel size, we use $F = 100$ filters. This multi-kernel CNN architecture enables the model to capture n-gram features at different granularities (3-grams, 4-grams, and 5-grams) without requiring explicit feature engineering.

RNN. To address the limitation of CNN's fixed receptive field (maximum 5-gram) in capturing long-range sequential dependencies, we incorporate a Bidirectional Long Short-Term Memory (BiLSTM) layer into the base architecture. We feed the output sequence of CNN into a BiLSTM layer with $H = 128$ hidden dimension. The BiLSTM processes the sequence in both forward and backward directions, generating a contextualized representation that captures comprehensive contextual information for each word token.

3. Experiment

3.1. Dataset

In this study, the Stanford Sentiment Treebank (SST-1)¹ dataset is employed to evaluate the performance of the proposed model. SST-1 is a widely used benchmark dataset for sentiment classification tasks and consists of movie review

¹<http://nlp.stanford.edu/sentiment/>

sentences annotated with fine-grained sentiment labels. The dataset is divided into training, validation, and test sets, containing a total of 11855 labeled movie review texts. Each sample is assigned to one of five sentiment categories: very positive, positive, neutral, negative, and very negative.

Before training, all texts are tokenized and converted into numerical representations according to the constructed vocabulary table. To ensure a fair comparison, the same data preprocessing procedures are applied to all experimental models. The training set (8544 samples) is used for parameter optimization, the validation set (1101 samples) is employed for hyperparameter tuning and model selection, and the test set (2210 samples) is utilized to evaluate the final classification performance.

3.2. Pre-trained Word Vectors

- **GloVe:** GloVe² [4] is an unsupervised learning algorithm that generates word embeddings by leveraging global word-word co-occurrence statistics from a corpus, capturing both semantic and syntactic relationships. Unlike local context-based methods, GloVe incorporates global corpus information into the embedding learning process, enabling semantically related words to obtain similar vector representations.
- **Word2Vec:** Word2Vec³ [1] is a neural-network-based word embedding model. It learns distributed word representations through either the Continuous Bag-of-Words (CBOW) architecture or the Skip-Gram architecture. CBOW predicts a target word from its surrounding context, whereas Skip-Gram predicts surrounding words given a target word. By training on large-scale unlabeled corpora, Word2Vec can capture both semantic and syntactic relationships among words.
- **FastText:** FastText⁴ [2] is an extension of Word2Vec proposed by Facebook AI Research. Unlike traditional word embedding methods that represent each word as a single vector, FastText represents a word as a collection of character n-grams and generates the final word vector by aggregating the vectors of its subword units. This mechanism enables FastText to better handle rare and out-of-vocabulary (OOV) words while capturing morphological information.
- **BERT:** BERT⁵ is a contextualized language representation model based on the Transformer architecture. Unlike static embeddings such as GloVe and

Models	Accuracy(%)
paper	48.70
Random	43.80
FastText	44.34
GloVe	46.79
Word2Vec	48.73

Table 1. Test accuracy obtained with different pre-trained word embeddings for embedding layer initialization under the same CNN architecture.

Word2Vec, BERT generates dynamic word representations according to the context in which a word appears. It is pre-trained using Masked Language Modeling (MLM) and Next Sentence Prediction (NSP) objectives on large-scale corpora, enabling it to capture deep bidirectional contextual information.

To ensure a fair comparison, the same CNN architecture and training settings are adopted for all experiments. The only difference among the models lies in the initialization of the embedding layer. The classification accuracies obtained using different pre-trained word embeddings are reported and compared in the subsequent experimental results section.

As for BERT, we employ the pre-trained bert-base-uncased model as our contextual encoder. For each input sentence, we tokenize it and feed the token sequence into BERT, extracting the last-layer hidden states for all tokens, which yields a sequence matrix of shape $(L, 768)$. This matrix is then directly fed into the classification head, which is built upon the CNN and/or RNN modules.

3.3. Metrics

To evaluate the performance of the proposed CNN-based sentiment classification models, accuracy is used as the primary evaluation metric, which measures the proportion of correctly classified samples among all test samples. The calculation formula is defined as follows:

$$\text{Accuracy} = \frac{N_{\text{correct}}}{N_{\text{total}}} \times 100\% \quad (1)$$

3.4. Experimental Results

As shown in Table 1, different pre-trained word embeddings on the classification performance of the baseline CNN architecture differ significantly. All three pre-trained variants outperform the random baseline, showing that transferring knowledge from large-scale corpora effectively enhances the discriminative capacity of the CNN model. Among the pre-trained word embeddings, Word2Vec achieves the highest classification accuracy,

²<https://nlp.stanford.edu/projects/glove/>

³<https://code.google.com/archive/p/word2vec/>

⁴<https://fasttext.cc/docs/en/crawl-vectors.html>

⁵<https://huggingface.co/google-bert/bert-base-uncased>

Models	Accuracy(%)		
	RNN	CNN	RNN + CNN
Random	43.03	43.80	42.41
FastText	44.57	44.71	44.62
GloVe	45.20	46.38	45.26
Word2Vec	45.61	48.55	48.92

Table 2. Ablation study on the impact of pre-trained word embeddings and network architectures on classification accuracy.

Models	Layer		Accuracy(%)
	RNN	CNN	
Baseline	✗	✗	50.95
Only RNN	✓	✗	51.31
Only CNN	✗	✓	52.04
RNN + CNN	✓	✓	52.85

Table 3. Ablation study on the impact of RNN and CNN components on classification accuracy with BERT pre-trained model.

demonstrating its superior performance in capturing semantic and syntactic relationships among words. Compared with the accuracy of 48.7% from the original paper, our results indicate that the CNN architecture combined with pre-trained word embeddings can achieve a comparable performance in sentiment classification tasks.

3.5. Ablation Study

To thoroughly investigate the contribution of each component in the proposed framework, we conduct a series of ablation experiments. The study targets two main axes: (i) the influence of different pre-trained static word embeddings and architectural choices (RNN, CNN, and their combination) and (ii) the marginal benefit of recurrent and convolutional layers when built on top of powerful contextualized representations from BERT.

As is shown in Table 2, the results obtained with different pre-trained word vectors initializations across three network variants: a pure BiLSTM (RNN), a pure multi-kernel CNN, and a joint RNN-CNN architecture. In the RNN-CNN architecture, we simplify the CNN part to a single convolutional layer with a kernel size of 5, and the RNN part to a single BiLSTM layer with 128 hidden units. The results show that the CNN architecture alone with multi-kernel convolutional layers achieves the highest accuracy, proving the advantage of convolutional layers in capturing local patterns in text data.

As shown in Table 3, we further evaluate the contribution of RNN and CNN components when initialized with the BERT pre-trained model. The result shows that the joint RNN-CNN architecture achieves the highest accuracy, in-

dicating that both recurrent and convolutional layers contribute to the performance improvement. It may prove that the combination of RNN and CNN can better process the feature output from BERT encoder, which can capture both local and global features of the input text.

4. Discussion

Overall, the experimental analysis confirms that the proposed combination of pre-trained embeddings and neural architectures can effectively improve sentiment classification performance. However, the results are also effected by the learning rates used in the training process, which are usually different for CNN and RNN. In the future, we plan to train the CNN and RNN layers with different learning rates to further improve the performance of the model.

5. Conclusion

In this work, we investigate CNN-based approaches for text sentiment classification and analyze the influence of different word representation methods and network architectures. We evaluate several pre-trained word embeddings, including FastText, GloVe, and Word2Vec, under the same CNN classification framework. Experimental results on the SST-1 dataset demonstrate that pre-trained embeddings significantly improve classification accuracy compared with random initialization, with Word2Vec achieving the best performance among static word representations.

Furthermore, we conduct extensive ablation experiments to analyze the contribution of different network components. The results show that CNN can effectively capture local sentiment patterns, while RNN provides additional capability for modeling sequential dependencies. By combining CNN and RNN, the model can integrate both local and global textual information, resulting in improved classification performance which is shown in the experiments of BERT.

References

- [1] Kenneth Ward Church. Word2vec. *Natural Language Engineering*, 23(1):155–162, 2017. 2
- [2] Edouard Grave, Piotr Bojanowski, Prakhar Gupta, Armand Joulin, and Tomas Mikolov. Learning word vectors for 157 languages. In *Proceedings of the International Conference on Language Resources and Evaluation (LREC 2018)*, 2018. 2
- [3] Yoon Kim. Convolutional neural networks for sentence classification. In *Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP)*, pages 1746–1751, 2014. 1
- [4] Jeffrey Pennington, Richard Socher, and Christopher D Manning. Glove: Global vectors for word representation. In *Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP)*, pages 1532–1543, 2014. 2